

TITLE PAGE

Project Name: ALUMNEX CONNECT
Subtitle: Comprehensive Full-Stack Software Engineering Project Report

Student Name: Sriman Kumar V
Register Number: [Insert Register Number]
Department: Computer Science and Engineering
Institution: [Insert Institution Name]
Guide: [Insert Guide Name]
Academic Year: 2026

CERTIFICATE

This is to certify that the project report entitled **ALUMNEX CONNECT** is a bonafide record of the work done by **Sriman Kumar V** in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering.

Signature of Guide _____
Signature of HOD _____

DECLARATION

I hereby declare that the project report entitled **ALUMNEX CONNECT** submitted to the institution is a record of original work done by me. This project has not been submitted in part or full to any other university or institution for the award of any degree or diploma.

Sriman Kumar V
Date: 2026-09-11

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all those who provided support and guidance during this project. I am thankful to my project guide for their continuous encouragement and insightful feedback. I also extend my thanks to the department and institution for providing the necessary resources and environment to successfully complete this endeavor.

ABSTRACT

Alumnex Connect is a comprehensive, full-stack, enterprise-grade web and mobile application designed to bridge the gap between students, alumni, and institutions. Built on the MERN stack (MongoDB, Express.js, React, Node.js) and enveloped in a Capacitor mobile wrapper, the platform facilitates professional networking, mentorship, career opportunities, and real-time communication.

The system distinguishes itself through advanced architectural implementations including AI-assisted career mentoring with provider failover (Groq to Gemini), real-time horizontal scaling capabilities via Socket.IO and Redis, automated developer statistics aggregation from platforms like GitHub and LeetCode, and a robust role-based access

control (RBAC) mechanism. The application enforces strong security measures including double-submit cookie CSRF protection, rate limiting, and strictly separated environments. This report provides deep, evidence-based documentation of the architecture, implementation, resilience mechanisms, and exact source code traceability.

TABLE OF CONTENTS

- 1. INTRODUCTION
- 2. SYSTEM STUDY AND REQUIREMENTS
- 3. SYSTEM ARCHITECTURE
- 4. TECHNOLOGY STACK
- 5. DATABASE DESIGN
- 6. AUTHENTICATION AND ACCESS CONTROL
- 7. SECURITY ARCHITECTURE
- 8. MODULE-WISE IMPLEMENTATION
- 9. AI FEATURES
- 10. REAL-TIME COMMUNICATION
- 11. EXTERNAL API INTEGRATIONS
- 12. PERFORMANCE AND OPTIMIZATION
- 13. TESTING AND QUALITY ASSURANCE
- 14. CI/CD AND DEPLOYMENT
- 15. MOBILE APPLICATION
- 16. FAILURE HANDLING AND RESILIENCE
- 17. THREAT MODEL AND SECURITY ANALYSIS
- 18. RESULTS AND OBSERVATIONS
- 19. LIMITATIONS
- 20. FUTURE ENHANCEMENTS
- 21. CONCLUSION APPENDICES

LIST OF ABBREVIATIONS

- **API**: Application Programming Interface
 - **JWT**: JSON Web Token
 - **RBAC**: Role-Based Access Control
 - **SPA**: Single Page Application
 - **CRUD**: Create, Read, Update, Delete
 - **CORS**: Cross-Origin Resource Sharing
 - **CSRF**: Cross-Site Request Forgery
 - **CSP**: Content Security Policy
 - **CDN**: Content Delivery Network
 - **TTL**: Time To Live
 - **CI/CD**: Continuous Integration / Continuous Deployment
 - **AI**: Artificial Intelligence
 - **NLP**: Natural Language Processing
 - **PDF**: Portable Document Format
 - **WebSocket**: Full-duplex communication protocol
 - **OTP**: One-Time Password
 - **2FA**: Two-Factor Authentication
 - **ATS**: Applicant Tracking System
-

CHAPTER 1 — INTRODUCTION

1.1 Background

Educational institutions inherently produce a massive network of alumni whose industry experience is highly valuable to current students. However, maintaining this connection post-graduation is traditionally difficult due to fragmented communication channels.

1.2 Problem Statement

Existing alumni networks are often passive directories. There is a lack of structured, actionable platforms where students can request mentorship, alumni can post internal job referrals, and real-time interaction can occur securely and reliably.

1.3 Motivation

To build an active, automated, and AI-enhanced ecosystem that incentivizes alumni engagement and provides students with data-driven career guidance, integrated directly with their developer profiles.

1.4 Objectives

- Implement a secure, RBAC-driven platform for multiple user roles (Admin, Student, Alumni, Mentor, Business).
- Provide real-time chat and notifications.
- Integrate AI tools to review resumes, simulate interviews, and match jobs.
- Aggregate external developer statistics (GitHub, LeetCode, etc.) into a unified profile.
- Ensure high resilience, fallback mechanisms, and security best practices.

1.5 Scope

The system encompasses a web SPA and a mobile application wrapper. It handles user authentication, profile management, mentorship lifecycles, job boards, AI interactions, and real-time messaging.

Implementation Evidence

Engineering Claim	Repository Evidence
MERN Stack	backend/server.js, frontend/package.json
Mobile Wrapper	mobile-wrapper/capacitor.config.json

CHAPTER 2 — SYSTEM STUDY AND REQUIREMENTS

2.1 Functional Requirements

- **Authentication:** Multi-provider login (Local, Google, GitHub), 2FA, OTP verification.
- **Mentorship:** Request flow, session tracking, mentor reviews.
- **AI Tools:** Resume parsing, mock interviews, AI career chat.
- **Networking:** Real-time messaging, forum posting, event management.

- **Developer Stats:** Automated fetching from GitHub, LeetCode, HackerRank, etc.

2.2 Non-Functional Requirements

- **Security:** Mitigation against XSS, CSRF, NoSQL injection, and brute-force attacks.
- **Resilience:** Graceful fallback if primary AI providers or Redis fails.
- **Performance:** Optimized API payloads, Redis caching for frequent queries, lazy loading in React.
- **Availability:** Load-balancer-aware API client with failover capabilities.

2.3 User Roles

- **Student:** Can apply for jobs, request mentorship, use AI tools.
 - **Alumni:** Can post jobs, become mentors, review resumes.
 - **Admin:** System configuration, content moderation, user management.
-

CHAPTER 3 — SYSTEM ARCHITECTURE

3.1 High-Level Architecture

The system follows a modern decoupled client-server architecture. The frontend is a React SPA communicating via RESTful APIs and WebSockets to an Express.js backend. The backend persists data in MongoDB and utilizes Redis for caching and WebSocket pub/sub scaling.

3.2 Frontend Architecture

- **Framework:** React with React Router for SPA navigation.
- **State Management:** React Query for server state caching/retries; React Context (`AuthContext` , `SocketContext`) for global client state.
- **API Client:** Axios instance (`api.js`) with interceptors for automatic token refresh, failover routing, and CSRF header injection.

3.3 Backend Architecture

- **Entry Point:** `server.js` chaining global middleware (Helmet, CORS, rate limits).
- **Routing Layer:** Express routers mapped to logical domains (`routes/auth.js` , `routes/ai.js`).
- **Service Layer:** Business logic separated from controllers (`services/authService.js`).
- **Data Access Layer:** Mongoose ODM with strictly defined schemas (`models/User.js`).

3.4 Resilience and Failover Architecture

The application implements client-side failover routing. If the primary backend URL fails (e.g., Redis sleep), the Axios interceptor shifts to a backup URL. On the backend, AI requests attempt Groq first, falling back to Gemini if rate-limited or unavailable.

```
graph TD
    Client[React / Capacitor Client] --> |HTTP/WS| LB{Client-side Failover}
    LB --> |Primary| API1[Primary API]
    LB --> |Backup| API2[Backup API]
    API1 --> Route[Express Router]
    Route --> AI{AI Orchestrator}
```

AI -->|Attempt 1| Groq[Groq API]
AI -->|Fallback| Gemini[Gemini API]

Implementation Evidence

Engineering Claim	Repository Evidence
Client-side API Failover	frontend/src/Utils/api.js lines 39-68
React Query Caching	frontend/src/App.js lines 66-84
AI Fallback Orchestrator	backend/Utils/aiHelper.js lines 73-85

CHAPTER 4 — TECHNOLOGY STACK

- **Frontend:** React, TailwindCSS, React Query, Socket.io-client. Selected for component reusability, efficient server state management, and rapid styling.
- **Backend:** Node.js, Express.js. Selected for high-concurrency non-blocking I/O, ideal for WebSocket chat and API aggregation.
- **Database:** MongoDB via Mongoose. Selected for flexible schema design accommodating diverse profile and developer statistics structures.
- **Caching/PubSub:** Redis. Selected for API caching and enabling horizontal scaling of Socket.IO instances.
- **Mobile:** Capacitor. Selected to wrap the web application into a native Android APK without maintaining a separate Kotlin/Swift codebase.
- **AI Providers:** Groq (Primary for speed), Gemini (Fallback).

CHAPTER 5 — DATABASE DESIGN

5.1 Models Overview

The system implements 23 distinct Mongoose models. Key models include:

- **User:** Core identity, authentication fields, role flags, profile metadata.
- **Mentorship:** Tracks the lifecycle of a mentor-mentee relationship (pending, active, completed, rejected).
- **Job:** Job postings with fields for ATS tracking, remote status, and required skills.
- **Message / ChatGroup:** Real-time communication persistence.

5.2 Schema Deep Dive: User

- **Important Fields:** email , password , role , isVerified , developerStats .
- **Security Considerations:** Passwords hashed via bcrypt. Excluded from default queries via select: false .

5.3 Entity Relationship Diagram

```
erDiagram
    USER ||--o{ MENTORSHIP : "requests / provides"
    USER ||--o{ JOB : "posts"
    USER ||--o{ JOB_APPLICATION : "applies"
```

```
USER ||--o{ MESSAGE : "sends"
JOB ||--o{ JOB_APPLICATION : "receives"
USER ||--o{ NOTIFICATION : "receives"
```

Implementation Evidence

Engineering Claim	Repository Evidence
23 Mongoose Models	<code>backend/models/</code> directory
Password hiding	<code>backend/models/User.js</code> (<code>select: false</code>)

CHAPTER 6 — AUTHENTICATION AND ACCESS CONTROL

6.1 Authentication Mechanisms

The system provides highly resilient, multi-strategy authentication:

- **Local Auth:** Email and hashed password. Returns a JWT.
- **OAuth:** Google and GitHub integrations via custom endpoints (`backend/routes/auth.js`).
- **2FA:** Email-based OTP verification during login if enabled.
- **Mobile Handling:** Custom deep-link handoffs and a specific `X-Mobile-App: capacitor` header to bypass cookie-based CSRF when using Bearer tokens.

6.2 Token Management

- **Web:** JWT is issued in an `HttpOnly` cookie to prevent XSS exfiltration.
- **Mobile/Fallback:** JWT is also issued in response bodies and stored in `localStorage` for the Capacitor app, sent via `Authorization: Bearer` headers.
- **Refresh Flow:** A 24-hour expiration window exists for token refresh (`backend/services/authService.js`).

6.3 Authorization Matrix

Derived directly from `backend/middleware/auth.js` (e.g., `authorize('alumni', 'mentor')`).

Capability	Student	Alumni	Admin
Request Mentorship	✓	—	—
Post Jobs	—	✓	✓
Access Admin Panel	—	—	✓
Access Forum	✓	✓	✓

Implementation Evidence

Engineering Claim	Repository Evidence
HttpOnly Cookie issuing	backend/services/authService.js (sendTokenResponse)
Token Refresh 24hr Window	backend/services/authService.js lines 393-413
RBAC Middleware	backend/middleware/auth.js (authorize function)

CHAPTER 7 — SECURITY ARCHITECTURE

7.1 Implemented Defenses

- **CSRF (Cross-Site Request Forgery):** Mitigated using a double-submit cookie pattern (backend/middleware/csrf.js). The client extracts XSRF-TOKEN and sends it in the X-XSRF-TOKEN header.
- **Rate Limiting:** Mitigates brute force. Global limits (express-rate-limit) applied in server.js .
- **Data Sanitization:** express-mongo-sanitize prevents NoSQL query injection.
- **XSS Mitigation:** Helmet.js sets strict Content Security Policies and X-XSS-Protection headers.
- **Error Disclosure:** The error handler in server.js strips stack traces when NODE_ENV !== 'development' .

7.2 Security Controls Matrix

Threat	Existing Mitigation	Residual Risk	Code Evidence
NoSQL Injection	express-mongo-sanitize removes \$ operators	Low	backend/server.js
CSRF	Double-submit cookie & X-Mobile-App bypass check	Low (Requires HTTPS)	backend/middleware/csrf.js
Brute Force	express-rate-limit applied globally	Medium (Distributed IPs)	backend/server.js
Information Leak	NODE_ENV check in error handler	Low	backend/server.js

CHAPTER 8 — MODULE-WISE IMPLEMENTATION

8.1 Mentorship Module

- **Workflow:** Student navigates to Mentorship page → Views active mentors → Clicks "Request" → Backend validates student role → Creates MentorshipRequest document → Triggers Socket.IO real-time notification to Mentor.
- **Edge Cases Handled:** Duplicate pending requests are blocked by the database and business logic.

8.2 Jobs and ATS Module

- **Workflow:** Alumni posts a job. Students apply.

- **AI Analysis:** A student can upload a PDF resume. The backend parses the PDF (`pdf-parse`), sends the text and Job Description to the AI orchestrator, and returns an ATS match score.
- **Edge Cases:** If the uploaded file is not a PDF, Multer/validation rejects it.

8.3 Developer Statistics Aggregation

- **Workflow:** A cron job or on-demand trigger fires `backend/utils/devStatsFetcher.js` .
- **Implementation:** The system aggregates data concurrently using `Promise.allSettled` from GitHub (GraphQL), LeetCode (GraphQL), HackerRank (REST), Codeforces, and Duolingo.
- **Fallback:** If GraphQL fails (e.g., missing token), GitHub fetching falls back to the REST API. If Cheerio scraping fails on GFG, it falls back to regex matching.

Implementation Evidence

Engineering Claim	Repository Evidence
Concurrent Stats Fetching	<code>backend/utils/devStatsFetcher.js</code>
Mentorship Models	<code>backend/models/Mentorship.js</code>

CHAPTER 9 — AI FEATURES

This chapter documents the integration of Large Language Models to simulate a career mentor, analyze resumes, and conduct mock interviews.

9.1 AI Architecture & Orchestration

The application implements an intelligent failover mechanism in `backend/utils/aiHelper.js` .

1. **Primary:** Groq (Llama-3/Compound-Mini) for ultra-fast inference.
2. **Fallback:** Gemini (Gemini-2.0-Flash). If Groq throws an error or rate limits, the `callAIWithFallback` orchestrator automatically routes the request to Gemini.
3. **Demo Mode:** If neither API key is configured (`process.env.GROQ_API_KEY` missing), the route explicitly returns a simulated/mock response (e.g., "I'm your AI Career Mentor! (Demo Mode)") instead of crashing.

9.2 ATS Resume Parsing (Verified Implementation)

- **Route:** `POST /api/ai/analyze-resume`
- **Mechanism:** `multer` receives the file in memory. `pdf-parse` extracts the raw text. The text and Job Description are injected into a strict system prompt demanding JSON output.
- **Handling:** The backend strips markdown (e.g., ``json) from the LLM output using regex before parsing.

Implementation Evidence

Engineering Claim	Repository Evidence
AI Failover Orchestrator	<code>backend/utils/aiHelper.js</code> (<code>callAIWithFallback</code>)
Demo Mode Detection	<code>backend/routes/ai.js</code> lines 22-31

CHAPTER 10 — REAL-TIME COMMUNICATION

10.1 Socket.IO Architecture

- **Frontend:** `SocketContext.js` maintains a singleton socket connection. It listens to the `backend-failover` event; if the HTTP client fails over to a backup server, the socket immediately reconnects to the new server URI.
- **Backend:** Handles presence (`user:online` , `user:offline`), messaging, and notifications.
- **Scaling:** Configured to use a Redis Adapter. This allows multiple Node.js instances to broadcast events seamlessly across instances.

```
sequenceDiagram
    participant C1 as Client A
    participant S1 as Socket Server 1
    participant R as Redis
    participant S2 as Socket Server 2
    participant C2 as Client B
    C1->>S1: Emit 'send_message'
    S1->>R: Publish to Redis Channel
    R->>S2: Message Event
    S2->>C2: Broadcast to Client B
```

Implementation Evidence

Engineering Claim	Repository Evidence
Client Socket Failover	frontend/src/contexts/SocketContext.js lines 58-90

CHAPTER 11 — EXTERNAL API INTEGRATIONS

Service	Purpose	Fallback / Failure Behavior	Status
Groq	Fast AI Inference	Falls back to Gemini	VERIFIED
Gemini	AI Inference Fallback	Returns mock demo response	VERIFIED
GitHub	Dev Stats / OAuth	GraphQL falls back to REST API	VERIFIED
LeetCode	Dev Stats	Returns null gracefully for module	VERIFIED
Cloudinary	Image/PDF Hosting	Validation errors returned to user	VERIFIED

CHAPTER 12 — PERFORMANCE AND OPTIMIZATION

- **Client-Side Lazy Loading:** React components inside `App.js` (e.g., `const Home = lazy(() => import('./pages/Home'))`) are lazy-loaded via `Suspense` to drastically reduce the initial JavaScript bundle size.
 - **React Query Caching:** Configured with a 2-hour `staleTime`. It prevents redundant API calls when navigating between tabs.
 - **Concurrent Scraping:** Developer statistics from 6 different platforms are fetched simultaneously using `Promise.allSettled` rather than sequentially, reducing latency.
-

CHAPTER 13 — TESTING AND QUALITY ASSURANCE

The repository contains automated tests executed via Jest and Supertest.

- **Test Files:** `auth.test.js`, `authService.test.js`, `jobService.test.js`, `mentorshipService.test.js`, `rbac.test.js`.
 - **Isolation:** Uses MongoDB in-memory server or a dedicated test database to ensure tests do not pollute production data.
-

CHAPTER 14 — CI/CD AND DEPLOYMENT

14.1 GitHub Actions Workflows

The `.github/workflows/` directory contains automation pipelines:

1. `test.yml`: Executes the Jest test suite on pushes to main branches.
2. `build-apk.yml`: Automatically builds the Capacitor Android APK and attaches it as a release artifact upon tagging.
3. `production-monitor.yml`: Automates health checks or ping monitoring.

14.2 Deployment Architecture

- **Frontend:** Can be hosted on Vercel/Netlify.
 - **Backend:** Hosted on Render/Heroku.
 - **Keep-Alive:** To combat Render's free-tier cold starts, the frontend Axios instance has a 60-second timeout, and the frontend load balancer automatically shifts URLs if a timeout occurs.
-

CHAPTER 15 — MOBILE APPLICATION

The project includes an Android application built using **Capacitor** (`mobile-wrapper/` directory).

- **Architecture:** It is a web-wrapper architecture, not a native Kotlin app. It bundles the React build into a native WebView.
 - **Authentication:** Because iOS/Android WebViews aggressively block cross-origin cookies (disabling standard CSRF protection), the system relies on `localStorage` to hold the JWT. The API client detects the Capacitor environment and injects the `Authorization: Bearer` and `X-Mobile-App: capacitor` headers to bypass strict cookie enforcement safely.
-

CHAPTER 16 — FAILURE HANDLING AND RESILIENCE

This system demonstrates immense engineering maturity through extensive fallback paths:

Failure Scenario	Detection	Fallback Action	Recovery
Primary Backend Offline	Axios interceptor catches timeout/error	Shifts baseURL to REACT_APP_BACKUP_API_URL	Triggers global event, swapping socket connections automatically
Groq API Fails	try/catch in aiHelper.js	Orchestrator calls Gemini API	Automatic next request
No AI Keys Configured	!process.env.GROQ_API_KEY	Hardcoded mock JSON/text returned	Requires admin environment configuration
Token Expiry (401)	Axios response interceptor	Puts requests in queue, silently calls /refresh	Replays pending requests if refresh succeeds
GitHub GraphQL Limit	GraphQL returns errors object	Drops back to GitHub v3 REST API	Resumes GraphQL when limit resets

CHAPTER 17 — THREAT MODEL AND SECURITY ANALYSIS

Residual Risks

- **Stored XSS:** If a user manages to inject script tags into a forum post and React's escaping fails, XSS is possible. Mitigated by Helmet CSP.
- **Account Takeover:** If the `JWT_SECRET` is compromised. Mitigated by a recent rotation to a 64-byte cryptographic hex string.
- **Capacitor Token Extraction:** Tokens in `localStorage` on mobile are theoretically accessible if the device is compromised, but this is a standard industry tradeoff for WebViews.

CHAPTER 18 — RESULTS AND OBSERVATIONS

The system successfully aggregates data from multiple developer platforms into unified profiles, demonstrating robust concurrent I/O. The failover routing on the client side proved highly effective during simulated backend outages, seamlessly shifting the Socket.IO and Axios connections to secondary endpoints without requiring a page reload.

CHAPTER 19 — LIMITATIONS

- **Configuration Dependent Features:** AI functionality is purely mocked unless Groq/Gemini API keys are provided.
- **Scraping Fragility:** The developer statistics fetcher for GeeksForGeeks and CodeChef relies on Cheerio DOM parsing, which will break if those platforms change their HTML structure.
- **WebRTC:** Video calling components exist (`VideoCallOverlay.js`) but rely heavily on standard mesh topologies which struggle with >4 participants.

CHAPTER 20 — FUTURE ENHANCEMENTS

1. **Queue-Based Background Jobs:** Move PDF parsing and AI processing to a BullMQ/Redis worker queue rather than keeping the HTTP request open.
2. **Distributed Tracing:** Implement Datadog or OpenTelemetry to observe failover events in production.
3. **Secret Rotation:** Implement automated 30-day rotation for JWT secrets and MongoDB credentials.

CHAPTER 21 — CONCLUSION

Alumnex Connect represents a mature, deeply defensive, and highly resilient full-stack application. By implementing layered security, automated failover mechanisms across HTTP, WebSockets, and AI providers, and delivering it via both Web and Capacitor-driven Mobile platforms, it solves the core problem of fragmented alumni networks. The source code serves as verifiable proof of advanced software engineering concepts including API load balancing, concurrent data aggregation, and strict role-based access control.

APPENDICES

Appendix A — Environment Variables

Variable	Used By	Required	Sensitive	Fallback
MONGODB_URI	Backend	Yes	Yes	Crash
JWT_SECRET	Backend	Yes	Yes	Crash
GROQ_API_KEY	Backend/AI	No	Yes	Gemini / Mock
GITHUB_TOKEN	Backend/Stats	No	Yes	REST API / Scrape

Appendix B — Feature-to-Code Traceability

Claim	Evidence File	Status
Multi-URL API Failover	frontend/src/utils/api.js	A. VERIFIED IMPLEMENTATION
AI Prompt Fallback	backend/utils/aiHelper.js	A. VERIFIED IMPLEMENTATION
Mobile CSRF Bypass	backend/middleware/csrf.js	A. VERIFIED IMPLEMENTATION

Capacitor Integration	mobile-wrapper/	A. VERIFIED IMPLEMENTATION
Redis WebSocket Adapter	backend/server.js / backend/utils/cache.js	F. INFERRED FROM CONFIG

END OF REPORT